SOUTHERN UNIVERSITY
LIFT CENTER LOGO

MBDA LOGO
(PER CURRENT BRANDING GUIDE)

PLATFORM SYSTEM DESIGN

Southern University LIFT Rural Business Center

Architecture and module specifications for the new and extended capabilities that close the identified platform gaps.

Program: MBDA Rural Business Center Program · MBDA-OBC-2026-00003
Recipient: Southern University and A&M College, College of Business
Prepared with: KDM & Associates, LLC — Program Management & Platform
Version: 1.0 Draft · October 25, 2026

Confidential draft for internal review. Logo placeholders will be replaced once the current MBDA branding guide is confirmed. This project is supported by the U.S. Department of Commerce, Minority Business Development Agency.

Contents

1. Architecture Overview

- 1.1 Deployment model
- 1.2 Module map
- 1.3 Design principles

2. Cross-Cutting Design

- 2.1 Roles and permissions (RBAC)
- 2.2 Unified client identity
- 2.3 Program event bus
- 2.4 Audit and records retention
- 2.5 Shared services

3. Module Specifications

- M1. LIFT Tenant, Branding & Feature Flags
- M2. Form Builder (Adams Enterprise AI Form Builder port)
- M3. AI Intake Interview (Requirements Interview Tool port)
- M4. Client Intake & Rural Eligibility
- M5. Client Lifecycle & Case Management
- M6. Referral & Service Routing
- M7. LIFT Learning Hub
- M8. Capital Readiness
- M9. Proposal Accelerator
- M10. Outcome Verification & Evidence Vault
- M11. MBDA Metrics & Reporting Engine
- M12. Grant Compliance & Invoice Tracker
- M13. Partner & Agreement Management
- M14. Feedback, Testimonials & Impact Stories
- M15. Program AI Assistant & Alerts

4. Data Model Summary

5. Integration Design

6. Security and Privacy

7. Delivery Phases

8. Open Decisions

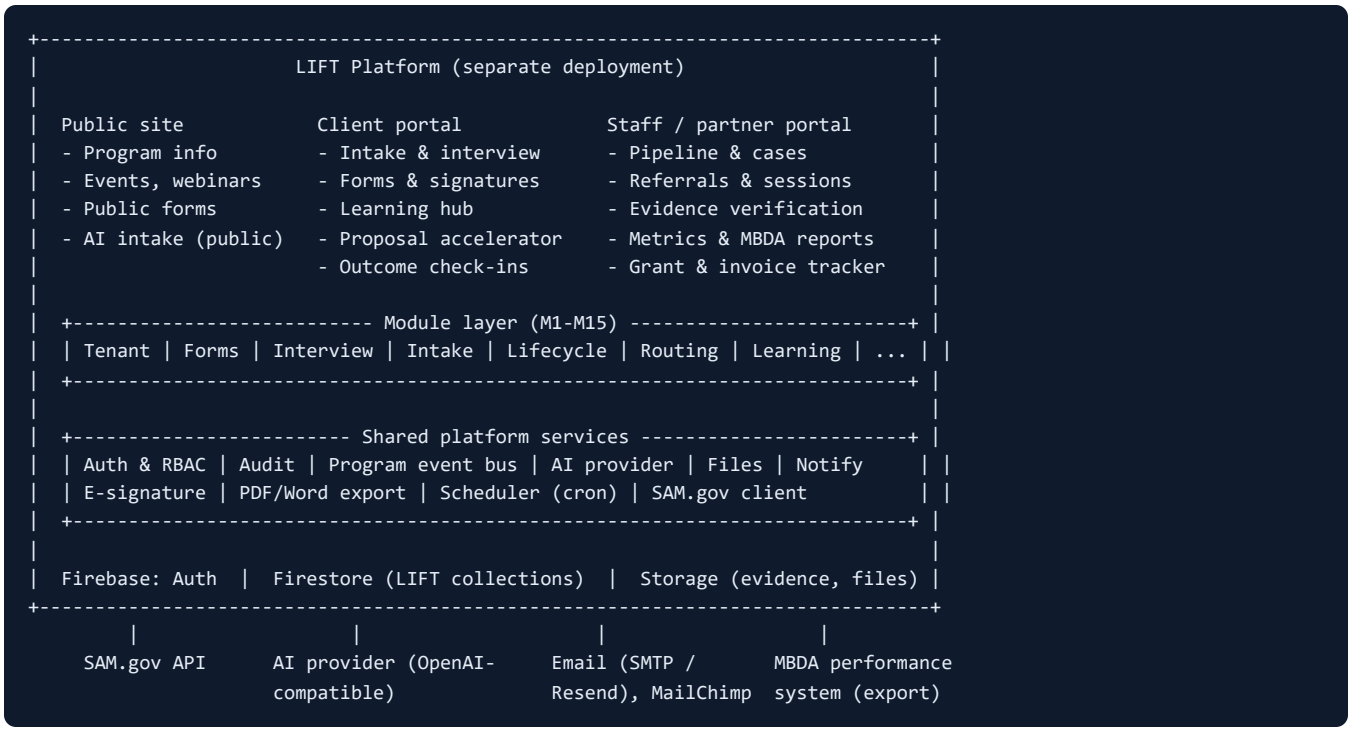
Program	LIFT Center — MBDA Rural Business Center Program (MBDA-OBC-2026-00003)
Document	System design for new and extended modules that close the gaps in `LIFT_Gap_Analysis.md`
Platform	`svp-platform` codebase — Next.js App Router, React, TypeScript, Firebase (Auth, Firestore, Storage), deployed as a **separate LIFT instance**
Version	1.0 — Draft for review
Date	October 25, 2026

1. Architecture Overview

1.1 Deployment model

The LIFT platform is a **separate deployment** of the existing `svp-platform` codebase:

- Its own Firebase project (Auth, Firestore, Storage), Vercel project, domain, and sending email address.
- LIFT/MBDA/Southern branding loaded from tenant configuration.
- KDM-only modules disabled by feature flags (see Gap Analysis §9).
- No LIFT client data is stored in the KDM consortium project. Any data sharing with KDM (for example, out-of-scope service referrals) requires client consent and a logged export.



1.2 Module map

ID	Module	Type	Primary gaps closed
M1	LIFT Tenant, Branding & Feature Flags	Extend white-label	Deployment, branding, module scope

ID	Module	Type	Primary gaps closed
M2	Form Builder (Adams port)	Port	Intake/consent forms, surveys, evidence forms, signatures
M3	AI Intake Interview (Requirements Interview Tool port)	Port	Conversational intake, baseline, check-ins
M4	Client Intake & Rural Eligibility	Extend client registration	MBDA LOE data, rural eligibility, enrollment
M5	Client Lifecycle & Case Management	Extend Kanbans/meetings	Stages, sessions, service plans
M6	Referral & Service Routing	Extend routing rules	Partner routing, non-duplication
M7	LIFT Learning Hub	Extend cohorts/events	Generic tracks, webinar-to-course, attendance
M8	Capital Readiness	New	Capital packages, lender pipeline, capital infusion
M9	Proposal Accelerator	Extend proposals	Real AI parsing, compliance matrix, drafting
M10	Outcome Verification & Evidence Vault	New	Evidence per milestone and outcome
M11	MBDA Metrics & Reporting Engine	New	KPIs, targets, roll-ups, MBDA export
M12	Grant Compliance & Invoice Tracker	New	Allowable costs, invoices, subaward chain
M13	Partner & Agreement Management	Extend strategic partners	MOU register, % executed
M14	Feedback, Testimonials & Impact Stories	Extend testimonials	Participant feedback, success stories
M15	Program AI Assistant & Alerts	Extend AI chat/notifications	Status queries, off-track alerts, briefings

1.3 Design principles

1. **Reuse before build.** Extend existing modules; port partner modules; build new only where nothing fits.
2. **Every metric is evidence-backed.** A number counts only when a verified outcome record with evidence supports it.
3. **One client, one ID.** A unified `liftClientId` follows the client across intake, services, partners, and outcomes.
4. **Humans decide; AI assists.** AI drafts, extracts, and flags; staff confirm eligibility, outcomes, and submissions.
5. **Least privilege.** Role- and field-scoped access to PII and financial data.
6. **Configuration over code.** Targets, forms, interview templates, stages, and routing rules are admin-editable.

2. Cross-Cutting Design

2.1 Roles and permissions (RBAC)

Add a `liftRole` claim (custom claim plus `liftUsers` document) alongside the existing `role` field.

Role	Description	Client PII	Financials	Evidence verify	Reports	Admin
`lift_admin`	Platform administrator (KDM)	Full	Full	Yes	All	Yes
`principal_investigator`	Dr. Mensah	Full	Full	Yes	All	Approve targets
`project_director`	Dr. Kaliba	Full	Full	Yes	All	Program config
`program_manager`	Center program managers	Assigned + team	Assigned	Yes	Program	No
`kdm_pmo`	KDM program office (Pam Ramos, Manpreet Hundal)	Full	Full	Yes	All	Compliance config
`partner_advisor`	SBDC, AM Lab, Law Center, Ag Center, GRS, V+ advisors	Referred clients only	Referred, if consented	Own services	Own	No
`instructor`	Workshop/cohort instructors	Enrolled participants (name, contact)	No	Attendance	Cohort	No
`evaluator`	Evaluation team / external reviewer	De-identified by default	Aggregated	No	All (read)	No
`client`	Business owner	Own	Own	Upload only	Own progress	No
`mbda_viewer`	MBDA program officer	Aggregated + sample records on request	Aggregated	No	MBDA reports (read)	No

Enforcement

- Server routes check `liftRole` with a shared `requireLiftRole()` helper (extending `lib/auth/server-auth.ts`).
- Firestore rules mirror role checks; sensitive subcollections (`financials`, `identity`) are server-only.
- Field-level masking in API responses for roles without PII access (for example, ethnicity and revenue).

2.2 Unified client identity

- `liftClients/{liftClientId}` is the system of record for a business.
- `liftClientContacts/{contactId}` stores people (owner, staff) linked to clients.
- Interviews, form submissions, sessions, referrals, cohort enrollments, proposals, and outcomes all store `liftClientId`.

- De-duplication on create: match by UEI, EIN (last 4 digits, hashed), normalized business name with parish, and owner email.

2.3 Program event bus

All modules emit domain events to `liftEvents`. Consumers (metrics, alerts, audit, notifications) subscribe through a server-side dispatcher.

```
Activity (session, attendance, award, form submission)
   -> liftEvents { type, liftClientId, payload, occurredAt }
      -> Outcome claim created (M10)        -> staff verifies with evidence
         -> Verified outcome                -> Metric fact (M11) for measure + period
            -> Dashboards, alerts (M15), MBDA report packs (M11)
```

Core event types: `client.created`, `client.enrolled`, `eligibility.decided`, `session.logged`, `referral.created`, `referral.completed`, `training.attended`, `cohort.completed`, `capital.package.completed`, `capital.obtained`, `proposal.submitted`, `contract.awarded`, `business.registered`, `outcome.verified`, `form.submitted`, `interview.completed`, `agreement.executed`.

2.4 Audit and records retention

- Extend the existing audit log with LIFT entity types; write-once audit entries for create, update, verify, export, sign, and delete actions.
- Retention: keep program records for at least 3 years after final financial report submission (2 CFR 200.334), unless the award specifies longer; legal hold flag.
- Exports (MBDA reports, data-sharing) are logged with the requester, scope, and file hash.

2.5 Shared services

Service	Implementation
AI provider	OpenAI-compatible adapter (`AI_BASE_URL`, `AI_API_KEY`, `AI_MODEL`) using the installed `openai` SDK; timeouts, structured output with Zod validation, deterministic fallbacks
Files	Firebase Storage with path-based rules: `lift/{liftClientId}/{category}/{fileId}`; SHA-256 hash stored with metadata
E-signature	Reuse the NDA signing flow (`/sign/nda/[token]`) and the Form Builder `signature` control; signature images in Storage
PDF/Word export	`puppeteer` (installed) for branded PDFs; `docx` (installed) for Word exports
Notifications	Existing `userNotifications`, email (SMTP/Resend), optional MailChimp/GoHighLevel campaigns
Scheduler	Vercel cron routes (pattern: `/api/cron/samgov-sync`) for reports, reminders, and check-ins

3. Module Specifications

M1. LIFT Tenant, Branding & Feature Flags

Purpose. Run the codebase as a LIFT-branded, separate deployment with only relevant modules enabled. Traces to REQ-Q1-03, REQ-X-06, REQ-X-14.

Data model

```
/** Singleton: liftTenantConfig/current */
export interface LiftTenantConfigDoc {
  programName: string; // "Louisiana Innovation for Thriving Rural Business (LIFT) Center"
  awardNumber: string; // "MBDA-OBC-2026-00003"
  periodStart: string; // "2026-09-01"
  periodEnd: string; // "2029-08-31"
  branding: {
    primaryLogoUrl: string; // Southern / LIFT
    mbdaLogoUrl: string; // per current MBDA branding guide
    mbdaLogoMinWidthPx: number; // enforce MBDA logo at least as prominent as Center logo
    colors: { primary: string; secondary: string; accent: string };
    fundingAcknowledgment: string; // required federal funding statement
    brandingGuideVersion?: string;
  };
  domains: { app: string; email: string };
  featureFlags: Record<LiftFeatureFlag, boolean>;
  updatedAt: Timestamp;
  updatedBy: string;
}

export type LiftFeatureFlag =
  | "consortiumMembership" | "affiliates" | "apollo" | "thomasnet" | "sourcewell"
  | "paidCohorts" | "outOfScopeServices" | "publicInterview" | "proposalAccelerator";
```

APIs. `GET/PATCH /api/lift/admin/tenant-config` (lift_admin). Middleware reads flags and returns 404 for disabled routes.

UI. `/portal/admin/lift/settings` (branding preview with MBDA logo-size check, feature flags, funding acknowledgment).

Acceptance criteria

- Disabled modules are unreachable by URL and hidden from navigation.
- Every public page renders the MBDA logo and funding acknowledgment per configuration.
- Branding changes are audited.

M2. Form Builder (Adams Enterprise AI Form Builder port)

Purpose. Configurable, signed, versioned forms that write to multiple LIFT records. Replaces JotForm. Traces to REQ-Q2-02, REQ-Q2-04, REQ-X-07.

Port scope (from spec §2)

Spec file	LIFT action
`lib/form-builder/types.ts`, `templates.ts`	Port; replace lookup sources and connectors

Spec file	LIFT action
`components/portal/form-builder-renderer.tsx`	Port; accessibility and low-bandwidth review
`app/(portal)/portal/form-builder/page.tsx`	Port under `/portal/admin/lift/forms`
`app/(marketing)/forms/[slug]/page.tsx`	Port as public form runner
`app/api/form-builder/import-document`, `ai-assist`	Port; route AI through shared provider
`app/api/public/forms/[slug]`	Port with security hardening below

Data model changes to the spec

```
export interface LiftFormDoc extends Omit<PlatformFormDoc, "apiKey"> {
  version: number; // increments on publish
  ombControlNumber?: string; // e.g., MBDA intake form
  ombExpiresOn?: string;
  consentText?: string; // exact text shown at signing
  fieldMappings: LiftFieldMapping[]; // multi-record writes
  audience: "public" | "client" | "staff" | "partner";
}

export interface LiftFieldMapping {
  fieldKey: string;
  target: "liftClients" | "liftClientContacts" | "liftBaselines" | "liftOutcomeClaims"
        | "liftSessions" | "liftCapitalPackages" | "liftAgreements";
  targetPath: string; // e.g., "business.naicsCodes"
  transform?: "none" | "currency_to_cents" | "split_csv" | "parish_lookup";
}

export interface LiftFormSubmissionDoc {
  id: string;
  formId: string;
  formVersion: number;
  liftClientId?: string;
  values: Record<string, unknown>; // non-file values only
  files: { fieldKey: string; storagePath: string; sha256: string; sizeBytes: number }[];
  signature?: {
    storagePath: string; // signature PNG in Storage, not Base64 in Firestore
    signedBy: string;
    signedAt: Timestamp;
    consentTextHash: string;
    contentHash: string; // SHA-256 of canonical values + form version
    ipAddress?: string;
    userAgent?: string;
  };
  source: "portal" | "public_link" | "api" | "interview_prefill";
  status: "submitted" | "accepted" | "rejected";
  createdAt: Timestamp;
}

/** Stored separately; never readable by clients */
export interface LiftFormApiKeyDoc { formId: string; keyHash: string; createdAt: Timestamp; revokedAt?: Timestamp }
```

Lookup sources (replace Adams sources): `lift_clients`, `parishes`, `service_areas`, `advisors`, `partners`, `cohorts`, `events`, `manual_list`.

Integrations: internal routes to M4, M5, M8, M10, M13 via field mappings; `outbound_webhook` (signed); email notification. Construction/field-service connectors are removed.

Security hardening (required)

- Replace `allow read: if true` on forms with a server endpoint returning a sanitized schema (no keys, no mappings).
- Store API keys hashed in `liftFormApiKeys`; compare by hash.
- Public submissions: Zod validation, rate limit, CAPTCHA (Turnstile) for public forms, file type/size limits, malware scan hook.
- Files and signatures in Storage; Firestore keeps references and hashes (avoids the 1 MiB document limit).

Dependencies. `recharts` and `pdf-parse` are installed. The spec's `jsPDF` is not installed; generate submission PDFs with the installed `puppeteer` to avoid a new dependency, or add `jspdf` after review. A `.docx` parser (for example `mammoth`) is required only for Word import.

APIs

- `GET /api/lift/forms/[slug]` (public, sanitized) · `POST /api/lift/forms/[slug]/submissions`
- `GET/POST/PATCH /api/lift/admin/forms` · `POST /api/lift/admin/forms/[id]/publish`
- `POST /api/lift/admin/forms/ai-assist` · `POST /api/lift/admin/forms/import-document`

Seed forms. MBDA Client Intake & LOE (document overlay), Rural Eligibility Attestation, Baseline Survey, Workshop Registration, Workshop Feedback, Outcome Evidence, Capital Readiness Checklist, Advisor Session Log, Partner Agreement Intake, Staff Onboarding Acknowledgment (catalog in Gap Analysis §8.3).

Acceptance criteria

- A signed MBDA intake submission creates or updates the client, contact, and consent records in one transaction.
- Each submission references the exact form version and consent text hash.
- No API key or field mapping is retrievable from a public endpoint.
- Submission PDF includes LIFT/MBDA branding, answers, signature, timestamp, and content hash.

M3. AI Intake Interview (Requirements Interview Tool port)

Purpose. Conversational, one-question-at-a-time intake, baseline, check-in, and debrief interviews that produce structured drafts for human confirmation. Traces to REQ-Q2-02, REQ-Q2-04, REQ-X-09, REQ-X-12.

Port scope. Port the portable core per the spec's package boundaries (§12): domain (`types`, `coverage`, `facts`, `specification`), services (`interview-engine`, `attachment-analyzer`, `output-generator`, `session-service`, `audit-service`), and ports/adapters (AI provider, Firestore, Firebase Auth, rate limiter, storage, notifier, action dispatcher). **Do not port** the PRD staging/deployment and module-build adapters (FR-15) to the LIFT site.

LIFT templates (configuration, FR-1)

Template slug	Audience	Output schema	Completion criteria
`lift-client-intake`	public + authenticated	`DraftIntakeRecord`, `NeedsAssessment`, `RecommendedServicePlan`	All required topics covered; name, business, parish, contact captured
`lift-baseline`	client	`BaselineSnapshot`	Revenue, employees, capital, contracts, digital tools captured
`lift-outcome-checkin`	client	`OutcomeClaim[]`	Each claimed outcome has amount/date and evidence request

Template slug	Audience	Output schema	Completion criteria
`lift-advisor-debrief`	staff, partner	`SessionLog`	Client, service area, duration, next steps
`lift-partner-intake`	partner	`PartnerProfile`	Services, eligibility, referral contact
`lift-platform-requirements`	staff	Spec's default specification	Spec's 15 topics

Categories (FR-2). LIFT service areas replace departments: Business Formation & Entrepreneurship; Capital Readiness; Procurement Readiness (CMMC/HUBZone/OZ/Mentor-Protégé); Advanced Manufacturing (AM Lab); Digital & AI Adoption; Agriculture & Rural Enterprise.

Data model (spec §9, LIFT additions)

```
export interface LiftInterviewDoc extends Interview {
  liftClientId?: string; // set when matched or created
  templateSlug: LiftInterviewTemplateSlug;
  draftOutputs: {
    intakeRecord?: DraftIntakeRecord;
    baseline?: BaselineSnapshot;
    outcomeClaims?: OutcomeClaimDraft[];
    servicePlan?: RecommendedServicePlan;
  };
  confirmation?: {
    status: "pending" | "confirmed" | "rejected";
    confirmedBy?: string; // staff uid or client uid
    confirmedAt?: Timestamp;
    formSubmissionId?: string; // signed consent from M2
  };
  aiDisclosureAcceptedAt?: Timestamp;
}
```

Post-completion actions (FR-17 adapters)

1. `interview.completed` → create draft client (M4) with `status: "intake_draft"`.
2. Prefill the MBDA Intake & LOE form (M2) with interview facts; send the signing link.
3. On signature → `client.enrolled` candidate; eligibility rule (M4) runs.
4. Recommended service plan → referrals (M6) after staff approval.
5. Outcome check-in → outcome claims (M10) with evidence requests.

Security & privacy (spec NFR-2, NFR-3, §15). Hashed public session tokens; HTTP-only cookie; rate limiting and CAPTCHA for public starts; prompt-injection handling (treat user and document text as data); AI disclosure before start; minimize data sent to AI (summaries, no full financial documents); never send SSNs/EINs to the AI provider (redact).

APIs (spec §10, namespaced)

- Public: `GET /api/lift/interview/templates/[slug]/categories`, `POST /api/lift/interview/templates/[slug]/sessions`, `POST /api/lift/interview/sessions/[id]/messages`, `POST .../attachments`, `GET .../status`
- Authenticated: `GET/POST /api/lift/interviews`, `GET/PATCH /api/lift/interviews/[id]`, `POST /api/lift/interviews/[id]/confirm`

- Admin: `GET/POST/PATCH /api/lift/admin/interview-templates`, `GET /api/lift/admin/interview-metrics`

UI. Public `/intake` (mobile-first chat), client portal `/portal/lift/interviews`, staff review queue `/portal/lift/intake-review`.

Acceptance criteria

- A public visitor can complete the intake interview without sign-in and receives a signing link for the MBDA form.
- No interview output becomes an enrolled client without confirmation and a signed consent submission.
- Coverage, facts, and output versions are audited with canonical hashes.
- When the AI provider is unavailable, the deterministic fallback completes the interview.

M4. Client Intake & Rural Eligibility

Purpose. The authoritative client record with MBDA LOE data, rural eligibility, and enrollment. Extends `ClientRegistrationDoc`. Traces to REQ-Q2-02, REQ-X-01.

Data model

```
export interface LiftClientDoc {
  id: string; // liftClientId
  legalName: string;
  dba?: string;
  status: LiftClientStage; // see M5
  enrollment: {
    enrolledAt?: Timestamp; // signed LOE + eligibility approved
    consentSubmissionId?: string; // M2 submission
    loeFormVersion?: string; // OMB control number + version
  };
  identifiers: { uei?: string; cageCode?: string; einLast4Hash?: string; samRegistered: boolean };
  location: { street: string; city: string; parish: string; state: string; zip: string; lat?: number; lng?: number
};
  eligibility: {
    rural: { status: "eligible" | "ineligible" | "review"; ruleId: string; source: string; decidedBy?: string;
decidedAt?: Timestamp };
    mbeAttestation: { attested: boolean; attestedAt?: Timestamp; certification?: string };
  };
  business: {
    stage: "pre_startup" | "startup" | "operating" | "growth";
    naicsCodes: string[];
    industry: string;
    employeesFullTime: number;
    employeesPartTime: number;
    annualRevenueBand: string;
    certifications: string[]; // 8(a), HUBZone, WOSB, SDVOSB, MBE, DBE
  };
  serviceAreas: LiftServiceArea[];
  assignedProgramManagerId?: string;
  source: "public_interview" | "form" | "kdm_pilot_import" | "referral" | "event";
  referralSource?: string;
  createdAt: Timestamp;
  updatedAt: Timestamp;
}

/** Server-only subcollection: liftClients/{id}/identity/private */
export interface LiftClientIdentityDoc { ownerEthnicity?: string; disadvantageBasis?: string; ownerPercentages?: {
name: string; pct: number }[] }
```

Rural eligibility rule engine

```
export interface LiftEligibilityRuleDoc {
  id: string;
  name: string; // e.g., "Rural parish list (confirm definition with MBDA)"
  type: "parish_list" | "census_tract" | "zip_list";
  ruralValues: string[]; // parish FIPS codes, tract IDs, or ZIPs
  effectiveFrom: string;
  source: string; // definition source, to be confirmed
  active: boolean;
}
```

- Parish lookup from address (geocoding or ZIP-to-parish table); rule decides `eligible`, `ineligible`, or `review`.
- Staff can override with a reason (audited).

KDM pilot import. One-time, consent-based import of Louisiana companies from KDM's database into `liftClients` with `source: "kdm_pilot_import"`; clients count only after signing the LIFT consent.

APIs. `GET/POST /api/lift/clients`, `GET/PATCH /api/lift/clients/[id]`, `POST /api/lift/clients/[id]/eligibility/decide`, `POST /api/lift/admin/import/kdm-pilot`.

Acceptance criteria

- Duplicate detection prevents two records for the same business.
- Eligibility decisions record the rule, source, decider, and time.
- Enrollment requires signed consent and an eligibility decision.

M5. Client Lifecycle & Case Management

Purpose. Pipeline stages, touchpoints/sessions, and service plans. Extends Readiness/Pipeline Kanbans and Meetings. Traces to REQ-Q4-01, REQ-X-09, REQ-X-10.

Stages

```
export type LiftClientStage =
  | "inquiry" | "intake_draft" | "intake_submitted" | "eligible" | "ineligible"
  | "assessed" | "service_plan" | "in_services" | "outcome_pending" | "outcome_verified"
  | "alumni" | "inactive";
```

Data model

```
export interface LiftSessionDoc {
  id: string;
  liftClientId: string;
  advisorId: string;
  partnerOrgId?: string;
  serviceArea: LiftServiceArea;
  type: "counseling" | "technical_assistance" | "assessment" | "training_followup";
  date: Timestamp;
  durationMinutes: number;
  notes: string; // restricted to staff roles
  nextSteps: string[];
  source: "manual" | "interview_debrief" | "form";
  createdAt: Timestamp;
}

export interface LiftServicePlanDoc {
  id: string;
  liftClientId: string;
  goals: { measure: LiftMeasureKey; description: string; targetDate?: string }[];
  services: { serviceArea: LiftServiceArea; provider: "center" | string; status: "planned" | "active" | "done" }[];
  approvedBy?: string;
  approvedAt?: Timestamp;
}
```

Rules. The first `session.logged` (or verified training attendance) marks the client as **served** for the period (Clients Served = unique clients with ≥1 session).

UI. LIFT Kanban (stage columns), client 360 page (profile, timeline, sessions, referrals, forms, outcomes, evidence), advisor "My clients".

Acceptance criteria. Stage changes are audited; a client appears once in Clients Served per reporting period; staleness flags clients with no touchpoint in 45 days.

M6. Referral & Service Routing

Purpose. Route clients to partners without duplicating services, and capture referral outcomes. Extends Lead Routing Rules. Traces to REQ-X-02.

```
export interface LiftReferralDoc {
  id: string;
  liftClientId: string;
  fromUserId: string;
  toPartnerId: string; // SBDC, AM Lab, Law Center, Ag Center, Entrepreneurship Institute, lender, APEX, GRS, V+
  serviceArea: LiftServiceArea;
  reason: string;
  consentToShare: boolean; // required for external partners
  sharedFields: string[];
  status: "proposed" | "sent" | "accepted" | "declined" | "completed";
  outcomeNote?: string;
  createdAt: Timestamp;
  updatedAt: Timestamp;
}
```

Routing rules. Extend `routingRules` with conditions on service area, parish, stage, and needs, and destinations of type partner; duplicate check blocks a referral if the client already receives the same service elsewhere.

Acceptance criteria. External referrals require client consent; partner advisors see only referred clients and shared fields; completed referrals can create outcome claims.

M7. LIFT Learning Hub

Purpose. Free, grant-funded tracks; webinar-to-course conversion; attendance feeding metrics. Extends Cohorts, Certificates, Events, and Webinars. Traces to REQ-Q2-03, REQ-Q3-02, REQ-X-11, REQ-X-16.

Changes

- Add `programTrack` to cohorts: `entrepreneurship`, `capital_readiness`, `am_business_program`, `cmmc_hubzone_oz_readiness`, `digital_ai_adoption`.
- Add `fundingType: "grant_free" | "fee_based"`; LIFT tracks default to `grant_free` with no checkout.
- **Webinar-to-course pipeline:** import an archived webinar (recording URL, slides, transcript) → AI-generated module outline, quiz, and summary → instructor review → publish with LIFT/MBDA branding.
- Attendance: event check-in and course completion emit `training.attended` and `cohort.completed`.

```
export interface LiftTrainingActivityDoc {
  id: string;
  title: string;
  type: "workshop" | "webinar" | "course" | "cohort_session";
  programTrack: LiftProgramTrack;
  deliveredAt: Timestamp;
  instructorIds: string[];
  partnerOrgIds: string[];
  attendeeClientIds: string[];
  attendanceEvidenceId?: string; // M10
  countsAsTrainingActivity: boolean;
}
```

Acceptance criteria. Each delivered workshop creates one training activity; attendees link to clients; completion certificates carry LIFT/MBDA branding and count toward AM and readiness completions.

M8. Capital Readiness

Purpose. Capital-readiness packages, lender/investor pipeline, and capital infusion outcomes. Traces to REQ-Q3-01, REQ-X-05.

```
export interface LiftCapitalPackageDoc {
  id: string;
  liftClientId: string;
  purpose: string;
  amountSoughtCents: number;
  capitalType: "loan" | "line_of_credit" | "equity" | "grant_external" | "microloan" | "other";
  checklist: { item: string; required: boolean; documentId?: string; complete: boolean }[];
  status: "in_progress" | "ready" | "submitted" | "approved" | "declined" | "withdrawn";
  completedAt?: Timestamp; // counts as a capital-readiness package
  lenderSubmissions: { lenderPartnerId: string; submittedAt: Timestamp; status: string }[];
}

export interface LiftCapitalObtainedDoc {
  id: string;
  liftClientId: string;
  packageId?: string;
  sourceName: string;
  sourceType: "bank" | "cdfi" | "sba_lender" | "investor" | "other";
  amountCents: number;
  closedAt: Timestamp;
  outcomeClaimId: string; // verified through M10
}
```

Guardrail. The platform records third-party capital only. A configuration check and UI copy prevent recording Center or award funds as loans, grants, or equity to clients (MOU §8).

Acceptance criteria. A package counts once when its checklist is complete; capital infusion counts only after verification with a closing document, commitment letter, or bank statement.

M9. Proposal Accelerator

Purpose. Help small businesses respond faster to SAM.gov RFIs, sources-sought notices, and RFPs. Replaces mock AI endpoints and extends the Proposals wizard. Traces to REQ-Q4-02, REQ-X-18.

Pipeline

```
SAM.gov opportunity (existing search + AI scoring)
  -> Ingest notice + attachments (SAM.gov resource links) -> text extraction (pdf-parse)
  -> AI solicitation analysis (replaces generateMockRfpProcessing)
  -> Go/No-Go analysis (replaces generateMockBidAnalysis) + teaming suggestions (lib/samgov-ai.ts)
  -> Compliance matrix (requirements, Section L instructions, Section M factors)
  -> Draft response from the capability library
  -> Advisor review queue -> client approval
  -> Export (docx / PDF) -> client submits -> submission recorded
  -> Award -> outcome claim (M10) -> Contracts Secured (M11)
```

Data model

```
export interface LiftSolicitationAnalysisDoc {
  id: string;
  noticeId: string; // SAM.gov
  solicitationNumber?: string;
  noticeType: "rfi" | "sources_sought" | "presolicitation" | "rfp" | "rfq" | "other";
  sourceDocuments: { name: string; storagePath: string; sha256: string; pages: number }[];
  extracted: {
    keyDates: { responseDue?: string; questionsDue?: string };
    naicsCodes: string[];
    setAside?: string;
    placeOfPerformance?: string[];
    evaluationFactors: { factor: string; weight?: number; description: string }[];
    submissionInstructions: { pageLimit?: number; format?: string; volumes?: string[] };
    mandatoryCertifications: string[]; // e.g., CMMC level, ISO 9001, AS9100
  };
  model: string;
  createdAt: Timestamp;
}

export interface LiftComplianceMatrixRowDoc {
  id: string;
  analysisId: string;
  requirementId: string; // e.g., "L.3.2" or "PWS 4.1"
  text: string;
  type: "shall" | "instruction" | "evaluation" | "question";
  responseSectionId?: string;
  ownerUserId?: string;
  status: "open" | "drafted" | "reviewed" | "complete";
}

export interface LiftCapabilityLibraryDoc {
  liftClientId: string;
  overview: string;
  coreCompetencies: string[];
  differentiators: string[];
  pastPerformance: { customer: string; contractNumber?: string; value?: number; period: string; description: string;
verified: boolean }[];
  certifications: { name: string; number?: string; expiresOn?: string; verified: boolean }[];
  keyPersonnel: { name: string; role: string; summary: string }[];
  updatedAt: Timestamp;
}

export interface LiftProposalResponseDoc {
  id: string;
  liftClientId: string;
  analysisId: string;
  responseType: "rfi_response" | "sources_sought" | "capability_statement" | "rfp_volume";
  sections: { id: string; title: string; content: string; aiDrafted: boolean; reviewed: boolean }[];
  reviewer?: { userId: string; reviewedAt: Timestamp; notes?: string };
  status: "drafting" | "in_review" | "client_approved" | "submitted" | "awarded" | "not_awarded";
  submittedAt?: Timestamp;
  timeToDraftMinutes?: number;
}
```

AI rules

- Drafts cite only capability-library facts; unverified claims are highlighted for the client to confirm.
- Never send CUI or export-controlled documents to the AI provider; a document classification prompt asks the user to confirm before upload.
- Every AI section is labeled `aiDrafted` until reviewed.

APIs. Replace the bodies of `/api/ai/rfp-processing` and `/api/ai/bid-analysis` with real implementations behind new LIFT routes: `POST /api/lift/proposals/ingest`, `POST /api/lift/proposals/[analysisId]/analyze`, `POST /api/lift/proposals/[analysisId]/compliance-matrix`, `POST /api/lift/proposals/responses`, `POST /api/lift/proposals/responses/[id]/draft-section`, `POST /api/lift/proposals/responses/[id]/export`.

Acceptance criteria. No mock data is returned; each compliance-matrix row links to the source page; exported Word documents respect page limits; awards create outcome claims.

M10. Outcome Verification & Evidence Vault

Purpose. Evidence-backed outcomes and milestone artifacts. Traces to all Year 1 verification methods.

```
export type LiftMeasureKey =
  | "clients_served" | "new_business_starts" | "capital_infusion" | "jobs_supported" | "revenue_growth"
  | "training_activities" | "capital_packages" | "contracts_secured" | "digital_adoption"
  | "am_participants" | "readiness_completions"
  | "personnel_onboarded" | "partner_agreements_executed" | "platform_operational" | "baseline_captured";

export interface LiftOutcomeClaimDoc {
  id: string;
  liftClientId?: string; // null for program-level milestones
  measure: LiftMeasureKey;
  value: number; // count, cents, jobs, or percentage
  occurredAt: Timestamp; // date of outcome (award date, loan closing, registration)
  attribution: string; // how the Center's assistance contributed
  evidenceIds: string[];
  status: "claimed" | "needs_info" | "verified" | "rejected";
  verifiedBy?: string;
  verifiedAt?: Timestamp;
  rejectionReason?: string;
  source: "staff" | "client_checkin" | "interview" | "form" | "system";
}

export interface LiftEvidenceDoc {
  id: string;
  type: "personnel_record" | "signed_mou" | "platform_test_report" | "onboarding_log" | "crm_report"
      | "attendance_record" | "pre_assessment_survey" | "loan_package" | "cohort_certificate"
      | "business_registration" | "contract_document" | "financial_statement" | "payroll_record"
      | "self_report_attestation" | "annual_report";
  storagePath: string;
  sha256: string;
  uploadedBy: string;
  uploadedAt: Timestamp;
  liftClientId?: string;
  confidential: boolean;
  retentionUntil: string;
}
```

Evidence requirements by measure (configurable)

Measure	Accepted evidence
New Business Starts	Business registration (Louisiana Secretary of State) dated after enrollment
Capital Infusion	Closing document, commitment letter, or bank statement
Jobs Supported	Payroll record or signed self-report attestation
Revenue Growth	Pre/post financial statements or tax returns (12-month comparison)

Measure	Accepted evidence
Contracts Secured	Award notice or contract document
Training / Certificates	Attendance record or certificate (auto-generated)
Q1 milestones	Personnel records, signed MOUs, platform test reports, onboarding logs

Workflow. Claim → evidence attached → reviewer verifies (role `program_manager`, `project_director`, or `kdm_pmo`) → `outcome.verified` → metric fact. Reviewer cannot verify their own claim.

Acceptance criteria. Unverified claims never appear in MBDA totals; every verified value links to at least one evidence item; evidence files are hash-verified on export.

M11. MBDA Metrics & Reporting Engine

Purpose. Compute core and project measures from verified facts against configurable targets; generate internal and MBDA report packs. Traces to REQ-Q2-V1, REQ-Q4-04, REQ-X-08.

```
export interface LiftMeasureDefinitionDoc {
  key: LiftMeasureKey;
  label: string;
  type: "core" | "project" | "milestone";
  unit: "count" | "usd" | "jobs" | "percent";
  aggregation: "unique_clients" | "sum" | "count" | "rate_annual";
  definition: string;
  evidenceTypes: LiftEvidenceDoc["type"][];
}

export interface LiftTargetDoc {
  measure: LiftMeasureKey;
  programYear: 1 | 2 | 3;
  quarterly: [number, number, number, number];
  annual: number;
  source: string; // e.g., "08/31 worksheet" — conflicts documented
  approvedBy?: string;
  approvedAt?: Timestamp;
}

export interface LiftMetricFactDoc {
  measure: LiftMeasureKey;
  liftClientId?: string;
  value: number;
  occurredAt: Timestamp;
  programYear: number;
  quarter: 1 | 2 | 3 | 4;
  outcomeClaimId: string;
}
```

Calculations

- **Clients Served:** unique `liftClientId` with ≥1 session or verified training attendance in the period; cumulative de-duplicated count across the program year.
- **Semi-annual assessment:** compare actuals to the sum of Q1–Q2 and Q3–Q4 targets, and to straight-line interpolation of annual targets; apply configurable seasonality weights.

- **Revenue Growth %:** share of clients with a 12-month baseline and follow-up who show an increase; reported annually.
- Program quarters follow the award period (Q1 = Sep–Nov 2026).

Reports

Report	Cadence	Audience	Contents
Program dashboard	Real time	Staff	Actual vs target by measure, pipeline by stage, at-risk clients
Monthly internal report	Monthly	PI, Project Director, KDM	Measures, activities, risks, corrective actions
Quarterly invoice support	Quarterly	Sponsored Programs	Activities delivered and evidence index (feeds M12)
Semi-annual MBDA report pack	Semi-annual	MBDA	Core and project measures, narrative, evidence index
Annual performance report	Annual	MBDA, Evaluation Team	Year results, validation, lessons learned

Export adapter. `MbdaExportAdapter` interface with CSV/XLSX and PDF implementations; a direct integration is added once the MBDA designated performance system and format are confirmed.

Acceptance criteria. Dashboard totals reconcile with verified facts; reports are reproducible for a given cutoff date (snapshot stored with hash); targets can change only with approval and are versioned.

M12. Grant Compliance & Invoice Tracker

Purpose. Track allowable activities, costs, and quarterly invoices across the subaward chain. Traces to REQ-X-04, REQ-X-05, REQ-X-15.

```
export interface LiftSubawardDoc {
  id: string;
  party: string; // KDM & Associates; sub-tier: V+, Be Wealthy With Pamela
  parentSubawardId?: string;
  poNumber?: string;
  status: "requested" | "in_review" | "executed";
  periodStart: string;
  periodEnd: string;
  amountCents: number;
  fiscalYearNote?: string;
}

export interface LiftInvoiceDoc {
  id: string;
  subawardId: string;
  period: { start: string; end: string; quarter: string };
  lineItems: { category: string; description: string; amountCents: number; allowable: boolean; receiptEvidenceIds:
string[] }[];
  deliverableEvidenceIds: string[]; // activities and milestones from M10/M11
  status: "draft" | "internal_review" | "submitted" | "approved" | "paid" | "returned";
  submittedTo: string; // KDM or Southern Sponsored Programs
  submittedAt?: Timestamp;
  totalCents: number;
}
```

Rules. Block invoice submission when the subaward or PO is not executed; flag unallowable categories (direct loans, grants, equity investments, construction, lobbying); require receipts for each line item.

Acceptance criteria. An invoice packet exports as PDF with line items, receipts index, and deliverables evidence; status history is audited.

M13. Partner & Agreement Management

Purpose. Partner register and agreement status for the Q1 "80% partner agreements executed" milestone. Extends Strategic Partners. Traces to REQ-Q1-04.

```
export interface LiftPartnerDoc {
  id: string;
  name: string;
  type: "university_unit" | "lender" | "sbdc" | "apex" | "prime" | "oem" | "nonprofit" | "consultant" | "agency";
  servicesOffered: LiftServiceArea[];
  referralContact: { name: string; email: string; phone?: string };
  portalAccess: boolean;
}

export interface LiftAgreementDoc {
  id: string;
  partnerId: string;
  type: "mou" | "subaward" | "data_sharing" | "letter_of_commitment";
  required: boolean; // counts toward the 80% milestone
  status: "draft" | "sent" | "partially_signed" | "executed" | "expired" | "terminated";
  signedDocumentEvidenceId?: string;
  executedAt?: Timestamp;
  expiresAt?: Timestamp;
}
```

Metric. `partner_agreements_executed = executed required agreements / total required agreements`.

M14. Feedback, Testimonials & Impact Stories

Purpose. Participant feedback and approved success stories. Extends `testimonials`. Traces to REQ-X-17.

- Feedback via M2 forms (NPS, ratings, impact questions) linked to the training activity or session.
- Impact story workflow: draft → client consent to publish → communications review → publish (with MBDA branding rules).
- Aggregated feedback feeds the monthly report and adaptive management.

M15. Program AI Assistant & Alerts

Purpose. Ask the platform about any client or measure; receive off-track alerts and scheduled briefings. Extends `/api/ai/chat` and notifications. Traces to REQ-X-09, REQ-X-10.

- **Retrieval tools (server-side, role-scoped):** `getClientSummary(liftClientId)`, `getMeasureProgress(measure, period)`, `listAtRiskClients()`, `getUpcomingDeadlines()`. The model receives only data the requesting user may see.
- **Alerts:** measure behind interpolated target by more than a configurable threshold; client stale for 45 days; evidence pending more than 14 days; invoice due within 10 days; agreement expiring within 30 days.

- **Briefings:** weekly email digest to leadership; optional Telegram channel using the existing Hermes operations-agent pattern.

4. Data Model Summary

Collection	Module	Key relationships
`liftTenantConfig`	M1	Singleton
`liftUsers`	Cross-cutting	User ↔ `liftRole`, partner org
`liftForms`, `liftFormSubmissions`, `liftFormApiKeys`	M2	Submission → client, consent, evidence
`liftInterviewTemplates`, `liftInterviews`, `liftInterviewOutputs`	M3	Interview → client, form prefill
`liftClients` (+ `identity`, `financials` server-only subcollections), `liftClientContacts`, `liftEligibilityRules`	M4	Client hub
`liftSessions`, `liftServicePlans`	M5	→ client, advisor, partner
`liftReferrals`	M6	→ client, partner
`liftTrainingActivities` (+ existing `cohorts`, `events`, `webinars`)	M7	→ attendees (clients)
`liftCapitalPackages`, `liftCapitalObtained`	M8	→ client, lender, outcome claim
`liftSolicitationAnalyses`, `liftComplianceMatrixRows`, `liftCapabilityLibraries`, `liftProposalResponses`	M9	→ client, SAM.gov notice
`liftOutcomeClaims`, `liftEvidence`, `liftBaselines`	M10	→ client, measure
`liftMeasureDefinitions`, `liftTargets`, `liftMetricFacts`, `liftReportSnapshots`	M11	→ outcome claims
`liftSubawards`, `liftInvoices`	M12	→ evidence
`liftPartners`, `liftAgreements`	M13	→ evidence
`liftFeedback` (+ existing `testimonials`)	M14	→ client, activity
`liftEvents`, `liftAlerts`	Cross-cutting, M15	Event log

Collection constants are added to `COLLECTIONS` in `lib/schema.ts` with a `LIFT_` prefix (for example, `LIFT_CLIENTS: "liftClients"`). All documents include `createdAt` / `updatedAt` using `Timestamp.now()`.

5. Integration Design

Integration	Use	Notes
SAM.gov API	Opportunity search, notices, attachments	Existing client and sync cron; add attachment download
AI provider (OpenAI-compatible)	Interview, form AI assist, proposal analysis/drafting, assistant	Approved model and data-handling terms required
Email (SMTP/Resend)	Notifications, signing links, reports	LIFT sending domain
MailChimp / GoHighLevel	Outreach campaigns	Optional; consent-based lists only
Calendar / bookings	Advisor sessions, workshops	Existing availability and bookings
Firebase Storage	Evidence, signatures, attachments	Path-based security rules
Stripe	Out-of-scope, fee-based services only	Disabled for grant-funded services
MBDA performance system	Report export	Adapter pending format confirmation
Partner systems (SBDC, lenders)	Referrals	Email/webhook initially; API later

6. Security and Privacy

- **Data classification:** Public, Internal, Confidential (business/financial), Restricted (identity, disadvantage basis, tax IDs).
- **Encryption:** TLS in transit; Firebase encryption at rest; hashed or last-4 storage for EIN; no SSNs collected.
- **Consent:** MBDA privacy disclosure and client consent captured and versioned; separate consent for external referrals and for publishing impact stories.
- **AI data handling:** redact Restricted fields before prompts; no CUI or export-controlled documents; provider terms must prohibit training on client data.
- **Access reviews:** quarterly role review; partner access expires with the agreement.
- **FERPA:** student interns follow Southern's policies; student records are not stored in client records.
- **Data-sharing agreements:** required with each partner receiving client data.
- **Incident response:** documented contacts at Southern IT and KDM; audit log review.

7. Delivery Phases

Complexity tiers: **S** (configuration or small extension), **M** (moderate extension or straightforward port), **L** (new module or complex port).

Phase	Window	Modules (tier)	Exit criteria
Phase 0 — Pilot readiness	Now → Nov 10	M1 (S); M2 intake/consent form only (M); M4 basic client + parish rule (M); M7 webinar archive rebrand (S)	Pilot companies sign consent and are recorded; Q1 evidence exists
Phase 1 — Public launch	→ ~Nov 19	M3 (L); M5 (M); M6 (M); M13 (S); M15 basic (M)	Public intake live; pipeline and partner tracking in use
Phase 2 — Q2 operations	Months 4–6	M8 (M); M9 (L); M10 (L); M11 (L); M12 (M)	Evidence-backed dashboards; first semi-annual report pack
Phase 3 — Optimization	Months 7+	M14 (S); M15 alerts and briefings (M); enhancements	Adaptive management; sustainability content

8. Open Decisions

1. Authoritative Year 1–3 targets (Gap Analysis §2.3).
2. MBDA performance system format and reporting cadence.
3. Rural definition and eligibility data source.
4. Whether electronic signature satisfies MBDA's consent requirement for the OMB-approved intake form.
5. Approved AI provider/model and data-handling terms.
6. Partner portal access versus email-only referrals.
7. Data ownership and retention after August 31, 2029.
8. Out-of-scope services, fees, and invoicing party.
9. Whether to add `jspdf` and `mammoth`, or standardize on the installed `puppeteer` and `pdf-parse`.
10. LIFT domain name and sending email address.